# Vellore Institute of Technology (VIT)

## School of Computer Science and Engineering

### M.Tech Course: Machine Learning Lab (MACSE504)

### Case Study Based Mini-Project — Phase 1: Part A (10 Marks)

---

| **Student Name** | **Register Number**

| **Rohan** | **26MML0020**

---

In [ ]:
```
# ============================================================
# IMPORT LIBRARIES & CONFIGURE ENVIRONMENT
# ============================================================
import os
import sys
import warnings
warnings.filterwarnings("ignore")

# Numerical & Data Manipulation
import numpy as np
import pandas as pd
import scipy.stats as stats

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing & Model Selection utilities
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import (
    LabelEncoder,
    OneHotEncoder,
    StandardScaler
)

# Interactive Widgets for GUI Interface (Rubric Requirement)
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

# Configuration for aesthetic publication-ready plots
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.ava
plt.rcParams.update({
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
```

```
    'ytick.labelsize': 10,
    'legend.fontsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 120,
    'axes.edgecolor': '#cccccc',
    'axes.linewidth': 0.8
})

CHURN_PALETTE = {'No': '#2b5c8f', 'Yes': '#d9534f'}

print("✓ All libraries imported successfully.")
print(f"Pandas {pd.__version__} | NumPy {np.__version__} | Matplotlib {plt.matplotl
```

```
✓ All libraries imported successfully.
Pandas 2.3.3 | NumPy 1.26.4 | Matplotlib 3.10.0 | Seaborn 0.13.2
```

---

# 1. Dataset Ingestion & Systematic Structural Profiling

## Objective:

- Load the Telco Customer Churn dataset.
- Inspect dimensions ( `shape` ), head samples, memory usage, and full statistical descriptions.
- Screen for duplicate rows and null values.
- Conduct a complete unique values audit across all 21 columns.
- Categorize columns into numerical and categorical subsets.

In [ ]:
```
# ============================================================
# LOAD DATASET
# ============================================================
# Dataset path configuration (handles local environment and standard paths)
DATASET_PATH = "WA_Fn-UseC_-Telco-Customer-Churn.csv"
if not os.path.exists(DATASET_PATH):
    # Fallback to kaggle path if executing in Kaggle environment
    DATASET_PATH = "/kaggle/input/datasets/blastchar/telco-customer-churn/WA_Fn-Use

df = pd.read_csv(DATASET_PATH)
print("✓ Dataset Loaded Successfully from:", DATASET_PATH)
print("Dataset Shape (Rows, Columns):", df.shape)
print("\nFirst 5 Records:")
display(df.head())
```

```
✓ Dataset Loaded Successfully from: WA_Fn-UseC_-Telco-Customer-Churn.csv
Dataset Shape (Rows, Columns): (7043, 21)

First 5 Records:
```

| | customerID | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | MultipleL |
|---|---|---|---|---|---|---|---|---|
| **0** | 7590-VHVEG | Female | 0 | Yes | No | 1 | No | No ph se |
| **1** | 5575-GNVDE | Male | 0 | No | No | 34 | Yes | |
| **2** | 3668-QPYBK | Male | 0 | No | No | 2 | Yes | |
| **3** | 7795-CFOCW | Male | 0 | No | No | 45 | No | No ph se |
| **4** | 9237-HQITU | Female | 0 | No | No | 2 | Yes | |

5 rows × 21 columns

In [ ]:
```
# ============================================================
# BASIC INFORMATION & DATASET OVERVIEW
# ============================================================
print("--- DATASET INFO ---")
df.info()
print("\n" + "="*80 + "\n")

print("--- DESCRIPTIVE STATISTICS (ALL ATTRIBUTES) ---")
display(df.describe(include="all").T)
print("\n" + "="*80 + "\n")

print("--- DATA TYPES PER COLUMN ---")
print(df.dtypes)
print("\n" + "="*80 + "\n")

print("Rows           :", df.shape[0])
print("Columns        :", df.shape[1])
print("Duplicate Rows :", df.duplicated().sum())
print("\nMissing Values Summary:")
missing = df.isnull().sum().sort_values(ascending=False)
display(missing)
```

```
--- DATASET INFO ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

================================================================================

--- DESCRIPTIVE STATISTICS (ALL ATTRIBUTES) ---
```

| | count | unique | top | freq | mean | std | min | 25% | 50% |
|---|---|---|---|---|---|---|---|---|---|
| **customerID** | 7043 | 7043 | 7590-VHVEG | 1 | NaN | NaN | NaN | NaN | NaN |
| **gender** | 7043 | 2 | Male | 3555 | NaN | NaN | NaN | NaN | NaN |
| **SeniorCitizen** | 7043.0 | NaN | NaN | NaN | 0.162147 | 0.368612 | 0.0 | 0.0 | 0.0 |
| **Partner** | 7043 | 2 | No | 3641 | NaN | NaN | NaN | NaN | NaN |
| **Dependents** | 7043 | 2 | No | 4933 | NaN | NaN | NaN | NaN | NaN |
| **tenure** | 7043.0 | NaN | NaN | NaN | 32.371149 | 24.559481 | 0.0 | 9.0 | 29.0 |
| **PhoneService** | 7043 | 2 | Yes | 6361 | NaN | NaN | NaN | NaN | NaN |
| **MultipleLines** | 7043 | 3 | No | 3390 | NaN | NaN | NaN | NaN | NaN |
| **InternetService** | 7043 | 3 | Fiber optic | 3096 | NaN | NaN | NaN | NaN | NaN |
| **OnlineSecurity** | 7043 | 3 | No | 3498 | NaN | NaN | NaN | NaN | NaN |
| **OnlineBackup** | 7043 | 3 | No | 3088 | NaN | NaN | NaN | NaN | NaN |
| **DeviceProtection** | 7043 | 3 | No | 3095 | NaN | NaN | NaN | NaN | NaN |
| **TechSupport** | 7043 | 3 | No | 3473 | NaN | NaN | NaN | NaN | NaN |
| **StreamingTV** | 7043 | 3 | No | 2810 | NaN | NaN | NaN | NaN | NaN |
| **StreamingMovies** | 7043 | 3 | No | 2785 | NaN | NaN | NaN | NaN | NaN |
| **Contract** | 7043 | 3 | Month-to-month | 3875 | NaN | NaN | NaN | NaN | NaN |
| **PaperlessBilling** | 7043 | 2 | Yes | 4171 | NaN | NaN | NaN | NaN | NaN |
| **PaymentMethod** | 7043 | 4 | Electronic check | 2365 | NaN | NaN | NaN | NaN | NaN |
| **MonthlyCharges** | 7043.0 | NaN | NaN | NaN | 64.761692 | 30.090047 | 18.25 | 35.5 | 70.35 |
| **TotalCharges** | 7043 | 6531 | | 11 | NaN | NaN | NaN | NaN | NaN |
| **Churn** | 7043 | 2 | No | 5174 | NaN | NaN | NaN | NaN | NaN |

```
================================================================================

--- DATA TYPES PER COLUMN ---
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object

================================================================================

Rows           : 7043
Columns        : 21
Duplicate Rows : 0

Missing Values Summary:
customerID          0
DeviceProtection    0
TotalCharges        0
MonthlyCharges      0
PaymentMethod       0
PaperlessBilling    0
Contract            0
StreamingMovies     0
StreamingTV         0
TechSupport         0
OnlineBackup        0
gender              0
OnlineSecurity      0
InternetService     0
MultipleLines       0
PhoneService        0
tenure              0
Dependents          0
Partner             0
SeniorCitizen       0
Churn               0
dtype: int64
```

In [ ]:

```
# ============================================================
# UNIQUE VALUES AUDIT
# ============================================================
for col in df.columns:
    print("=" * 55)
    print(f"Column: {col}")
    print(f"Unique Count: {df[col].nunique()}")
    print("Sample Values:", df[col].unique()[:10])

# ============================================================
# COLUMN TYPES IDENTIFICATION
# ============================================================
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
categorical_cols = df.select_dtypes(include="object").columns.tolist()

print("\n" + "=" * 55)
print("Numerical Features (Pre-cleaning):")
print(numeric_cols)
print("\nCategorical Features (Pre-cleaning):")
print(categorical_cols)
```

```
==========================================================
Column: customerID
Unique Count: 7043
Sample Values: ['7590-VHVEG' '5575-GNVDE' '3668-QPYBK' '7795-CFOCW' '9237-HQITU'
 '9305-CDSKC' '1452-KIOVK' '6713-OKOMC' '7892-POOKP' '6388-TABGU']
==========================================================
Column: gender
Unique Count: 2
Sample Values: ['Female' 'Male']
==========================================================
Column: SeniorCitizen
Unique Count: 2
Sample Values: [0 1]
==========================================================
Column: Partner
Unique Count: 2
Sample Values: ['Yes' 'No']
==========================================================
Column: Dependents
Unique Count: 2
Sample Values: ['No' 'Yes']
==========================================================
Column: tenure
Unique Count: 73
Sample Values: [ 1 34  2 45  8 22 10 28 62 13]
==========================================================
Column: PhoneService
Unique Count: 2
Sample Values: ['No' 'Yes']
==========================================================
Column: MultipleLines
Unique Count: 3
Sample Values: ['No phone service' 'No' 'Yes']
==========================================================
Column: InternetService
Unique Count: 3
Sample Values: ['DSL' 'Fiber optic' 'No']
==========================================================
Column: OnlineSecurity
Unique Count: 3
Sample Values: ['No' 'Yes' 'No internet service']
==========================================================
Column: OnlineBackup
Unique Count: 3
Sample Values: ['Yes' 'No' 'No internet service']
==========================================================
Column: DeviceProtection
Unique Count: 3
Sample Values: ['No' 'Yes' 'No internet service']
==========================================================
Column: TechSupport
Unique Count: 3
Sample Values: ['No' 'Yes' 'No internet service']
==========================================================
Column: StreamingTV
Unique Count: 3
```

```
Sample Values: ['No' 'Yes' 'No internet service']
=======================================================
Column: StreamingMovies
Unique Count: 3
Sample Values: ['No' 'Yes' 'No internet service']
=======================================================
Column: Contract
Unique Count: 3
Sample Values: ['Month-to-month' 'One year' 'Two year']
=======================================================
Column: PaperlessBilling
Unique Count: 2
Sample Values: ['Yes' 'No']
=======================================================
Column: PaymentMethod
Unique Count: 4
Sample Values: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
=======================================================
Column: MonthlyCharges
Unique Count: 1585
Sample Values: [ 29.85  56.95  53.85  42.3   70.7   99.65  89.1   29.75 104.8   56.1
5]
=======================================================
Column: TotalCharges
Unique Count: 6531
Sample Values: ['29.85' '1889.5' '108.15' '1840.75' '151.65' '820.5' '1949.4' '301.
9'
 '3046.05' '3487.95']
=======================================================
Column: Churn
Unique Count: 2
Sample Values: ['No' 'Yes']

=======================================================
Numerical Features (Pre-cleaning):
['SeniorCitizen', 'tenure', 'MonthlyCharges']

Categorical Features (Pre-cleaning):
['customerID', 'gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSuppor
t', 'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMetho
d', 'TotalCharges', 'Churn']
```

---

# 2. Parsing 'TotalCharges', Data Type Rectification & Missing Value Handling

## Analysis of the Anomaly in `'TotalCharges'`:

1. In `df.info()`, `'TotalCharges'` is typed as `object` (string) rather than floating point currency.

2. A whitespace check reveals that **11 records** contain blank spaces ( `" "` ).
3. Cross-referencing with `'tenure'` shows that **every single one of these 11 customers has `tenure == 0`** (newly registered subscribers who have not yet had their first monthly billing cycle).
4. As per standard machine learning preprocessing protocol:
   - Drop the arbitrary identifier column `'customerID'` (zero predictive capacity).
   - Convert whitespace strings in `'TotalCharges'` to `np.nan` and cast to `float64` .
   - Drop the 11 unbilled records ( `df.dropna(inplace=True)` ), yielding an immaculate dataset of **7,032 rows and 20 columns**.

In [ ]:
```
# ============================================================
# TOTALCHARGES PARSING & DATA CLEANING
# ============================================================
# 1. Audit initial data type and whitespace count
print("Initial TotalCharges dtype:", df["TotalCharges"].dtype)
blank_count = (df["TotalCharges"] == " ").sum()
print(f"Count of blank whitespace strings in TotalCharges: {blank_count}")

# 2. Inspect the 11 records with blank TotalCharges
display(df[df["TotalCharges"] == " "][['customerID', 'tenure', 'MonthlyCharges', 'T

# 3. Drop unique identifier customerID
df.drop(columns=["customerID"], inplace=True)
print("✓ 'customerID' dropped")

# 4. Replace whitespace strings with NaN and cast to float
df["TotalCharges"] = df["TotalCharges"].replace(" ", np.nan)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"])

print(f"TotalCharges NaN count prior to drop: {df['TotalCharges'].isnull().sum()}")

# 5. Drop rows containing NaN (the 11 zero-tenure records)
df.dropna(inplace=True)
df.reset_index(drop=True, inplace=True)

# 6. Verify post-cleaning dimensions and null counts
print("\n--- Post-Cleaning Verification ---")
print("Cleaned DataFrame Shape :", df.shape)
print("Total Remaining Nulls   :", df.isnull().sum().sum())
print("TotalCharges Dtype      :", df["TotalCharges"].dtype)

# 7. Distribution of categories in object columns
print("\n--- Categorical Feature Distributions ---")
for col in df.select_dtypes(include="object").columns:
    print(f"\n--- {col} ---")
    print(df[col].value_counts())
```

```
Initial TotalCharges dtype: object
Count of blank whitespace strings in TotalCharges: 11
```

| | customerID | tenure | MonthlyCharges | TotalCharges | Contract | Churn |
|---|---|---|---|---|---|---|
| **488** | 4472-LVYGI | 0 | 52.55 | | Two year | No |
| **753** | 3115-CZMZD | 0 | 20.25 | | Two year | No |
| **936** | 5709-LVOEQ | 0 | 80.85 | | Two year | No |
| **1082** | 4367-NUYAO | 0 | 25.75 | | Two year | No |
| **1340** | 1371-DWPAZ | 0 | 56.05 | | Two year | No |
| **3331** | 7644-OMVMY | 0 | 19.85 | | Two year | No |
| **3826** | 3213-VVOLG | 0 | 25.35 | | Two year | No |
| **4380** | 2520-SGTTA | 0 | 20.00 | | Two year | No |
| **5218** | 2923-ARZLG | 0 | 19.70 | | One year | No |
| **6670** | 4075-WKNIU | 0 | 73.35 | | Two year | No |
| **6754** | 2775-SEFEE | 0 | 61.90 | | Two year | No |

```
✓ 'customerID' dropped
TotalCharges NaN count prior to drop: 11

--- Post-Cleaning Verification ---
Cleaned DataFrame Shape : (7032, 20)
Total Remaining Nulls   : 0
TotalCharges Dtype      : float64

--- Categorical Feature Distributions ---

--- gender ---
gender
Male      3549
Female    3483
Name: count, dtype: int64

--- Partner ---
Partner
No     3639
Yes    3393
Name: count, dtype: int64

--- Dependents ---
Dependents
No     4933
Yes    2099
Name: count, dtype: int64

--- PhoneService ---
PhoneService
Yes    6352
No      680
Name: count, dtype: int64

--- MultipleLines ---
MultipleLines
No                  3385
Yes                 2967
No phone service     680
Name: count, dtype: int64

--- InternetService ---
InternetService
Fiber optic    3096
DSL            2416
No             1520
Name: count, dtype: int64

--- OnlineSecurity ---
OnlineSecurity
No                     3497
Yes                    2015
No internet service    1520
Name: count, dtype: int64

--- OnlineBackup ---
```

```
OnlineBackup
No                     3087
Yes                    2425
No internet service    1520
Name: count, dtype: int64

--- DeviceProtection ---
DeviceProtection
No                     3094
Yes                    2418
No internet service    1520
Name: count, dtype: int64

--- TechSupport ---
TechSupport
No                     3472
Yes                    2040
No internet service    1520
Name: count, dtype: int64

--- StreamingTV ---
StreamingTV
No                     2809
Yes                    2703
No internet service    1520
Name: count, dtype: int64

--- StreamingMovies ---
StreamingMovies
No                     2781
Yes                    2731
No internet service    1520
Name: count, dtype: int64

--- Contract ---
Contract
Month-to-month    3875
Two year          1685
One year          1472
Name: count, dtype: int64

--- PaperlessBilling ---
PaperlessBilling
Yes    4168
No     2864
Name: count, dtype: int64

--- PaymentMethod ---
PaymentMethod
Electronic check             2365
Mailed check                 1604
Bank transfer (automatic)    1542
Credit card (automatic)      1521
Name: count, dtype: int64

--- Churn ---
```

```
Churn
No     5163
Yes    1869
Name: count, dtype: int64
```

---

# 3. Mandatory Part A Question 1 Analytics: Churn Proportions & Distributions

## Mandatory Sub-Tasks from Question 1:

1. **Analyze customer churn proportions across Contract Types** (*Month-to-month, One year, Two year*).
2. **Analyze customer churn proportions across Internet Service Types** (*DSL, Fiber optic, No*).
3. **Generate distribution plots for Tenure vs. Monthly Charges** and bivariate density heatmaps.
4. **Compute correlation matrices** for continuous numerical attributes with collinearity analysis.

In [ ]:
```
# ============================================================
# 3.1 CHURN PROPORTIONS ACROSS CONTRACT TYPES
# ============================================================
# Cross-tabulation: Contract vs Churn
contract_ct = pd.crosstab(df['Contract'], df['Churn'], margins=True, margins_name='
contract_pct = pd.crosstab(df['Contract'], df['Churn'], normalize='index') * 100

contract_summary = contract_ct.copy()
contract_summary['Churn Rate (%)'] = contract_pct['Yes']
print("Contract Type vs Churn Contingency Table:")
display(contract_summary.style.format({'No': '{:,}', 'Yes': '{:,}', 'Total': '{:,}'

# Chi-Square Test of Independence
chi2_c, p_c, _, _ = stats.chi2_contingency(pd.crosstab(df['Contract'], df['Churn']))
cramers_v_c = np.sqrt(chi2_c / (len(df) * (min(pd.crosstab(df['Contract'], df['Chur
print(f"\nChi-Square Test (Contract vs Churn): Chi2 = {chi2_c:.2f} | p-value = {p_c

# Visualizations: Contract Churn
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

# Subplot 1: Absolute Customer Count
sns.countplot(data=df, x='Contract', hue='Churn', palette=CHURN_PALETTE, ax=axes[0]
axes[0].set_title('Customer Count by Contract Type & Churn', fontweight='bold')
axes[0].set_xlabel('Contract Type')
axes[0].set_ylabel('Number of Customers')
axes[0].legend(title='Churn', frameon=True)
for p in axes[0].patches:
    h = p.get_height()
    if h > 0:
        axes[0].annotate(f'{int(h):,}', (p.get_x() + p.get_width() / 2., h / 2),
```

```
                          ha='center', va='center', color='white', fontsize=10, font

# Subplot 2: 100% Normalized Stacked Bar
contract_pct_sorted = contract_pct.loc[['Month-to-month', 'One year', 'Two year']]
contract_pct_sorted.plot(kind='bar', stacked=True, color=['#2b5c8f', '#d9534f'], ax
axes[1].set_title('100% Normalized Churn Proportion by Contract Type', fontweight='
axes[1].set_xlabel('Contract Type')
axes[1].set_ylabel('Percentage (%)')
axes[1].set_ylim(0, 105)
axes[1].tick_params(axis='x', rotation=0)
axes[1].legend(title='Churn', bbox_to_anchor=(1.02, 1), loc='upper left')

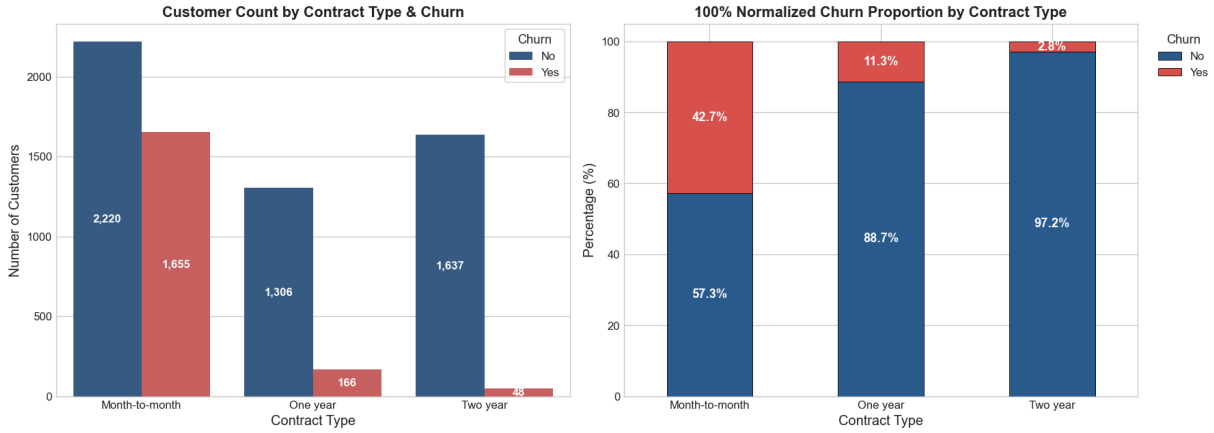
for i, (idx, row) in enumerate(contract_pct_sorted.iterrows()):
    axes[1].text(i, row['No'] / 2, f"{row['No']:.1f}%", ha='center', va='center', c
    axes[1].text(i, row['No'] + row['Yes'] / 2, f"{row['Yes']:.1f}%", ha='center',

plt.tight_layout()
plt.show()
```

```
Contract Type vs Churn Contingency Table:
```

| Churn | No | Yes | Total | Churn Rate (%) |
|---|---|---|---|---|
| **Contract** | | | | |
| **Month-to-month** | 2,220 | 1,655 | 3,875 | 42.71% |
| **One year** | 1,306 | 166 | 1,472 | 11.28% |
| **Two year** | 1,637 | 48 | 1,685 | 2.85% |
| **Total** | 5,163 | 1,869 | 7,032 | nan% |

```
Chi-Square Test (Contract vs Churn): Chi2 = 1179.55 | p-value = 7.3262e-257 | Cramé
r's V = 0.4096
```




In [ ]:

```
# ============================================================
# 3.2 CHURN PROPORTIONS ACROSS INTERNET SERVICE TYPES
# ============================================================
internet_ct = pd.crosstab(df['InternetService'], df['Churn'], margins=True, margins
internet_pct = pd.crosstab(df['InternetService'], df['Churn'], normalize='index') *

internet_summary = internet_ct.copy()
internet_summary['Churn Rate (%)'] = internet_pct['Yes']
print("Internet Service Type vs Churn Contingency Table:")
```

```
display(internet_summary.style.format({'No': '{:,}', 'Yes': '{:,}', 'Total': '{:,}'

# Chi-Square Test
chi2_i, p_i, _, _ = stats.chi2_contingency(pd.crosstab(df['InternetService'], df['C
cramers_v_i = np.sqrt(chi2_i / (len(df) * (min(pd.crosstab(df['InternetService'], d
print(f"\nChi-Square Test (Internet Service vs Churn): Chi2 = {chi2_i:.2f} | p-valu

# Visualizations: Internet Service Churn
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

sns.countplot(data=df, x='InternetService', hue='Churn', palette=CHURN_PALETTE, ax=
axes[0].set_title('Customer Count by Internet Service & Churn', fontweight='bold')
axes[0].set_xlabel('Internet Service Type')
axes[0].set_ylabel('Number of Customers')
axes[0].legend(title='Churn', frameon=True)
for p in axes[0].patches:
    h = p.get_height()
    if h > 0:
        axes[0].annotate(f'{int(h):,}', (p.get_x() + p.get_width() / 2., h / 2),
                         ha='center', va='center', color='white', fontsize=10, font

internet_pct_sorted = internet_pct.loc[['Fiber optic', 'DSL', 'No']]
internet_pct_sorted.plot(kind='bar', stacked=True, color=['#2b5c8f', '#d9534f'], ax
axes[1].set_title('100% Normalized Churn Proportion by Internet Service', fontweigh
axes[1].set_xlabel('Internet Service Type')
axes[1].set_ylabel('Percentage (%)')
axes[1].set_ylim(0, 105)
axes[1].tick_params(axis='x', rotation=0)
axes[1].legend(title='Churn', bbox_to_anchor=(1.02, 1), loc='upper left')

for i, (idx, row) in enumerate(internet_pct_sorted.iterrows()):
    axes[1].text(i, row['No'] / 2, f"{row['No']:.1f}%", ha='center', va='center', c
    axes[1].text(i, row['No'] + row['Yes'] / 2, f"{row['Yes']:.1f}%", ha='center',

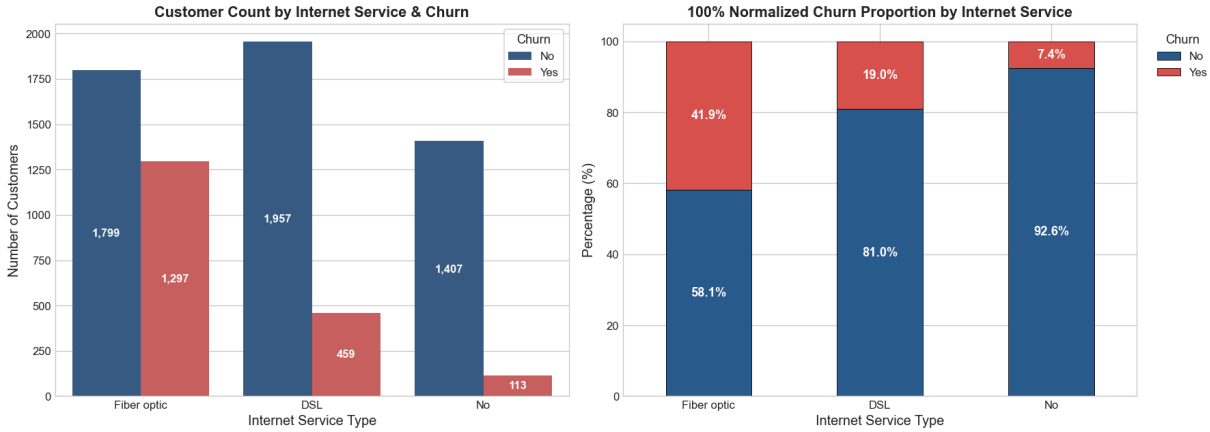
plt.tight_layout()
plt.show()

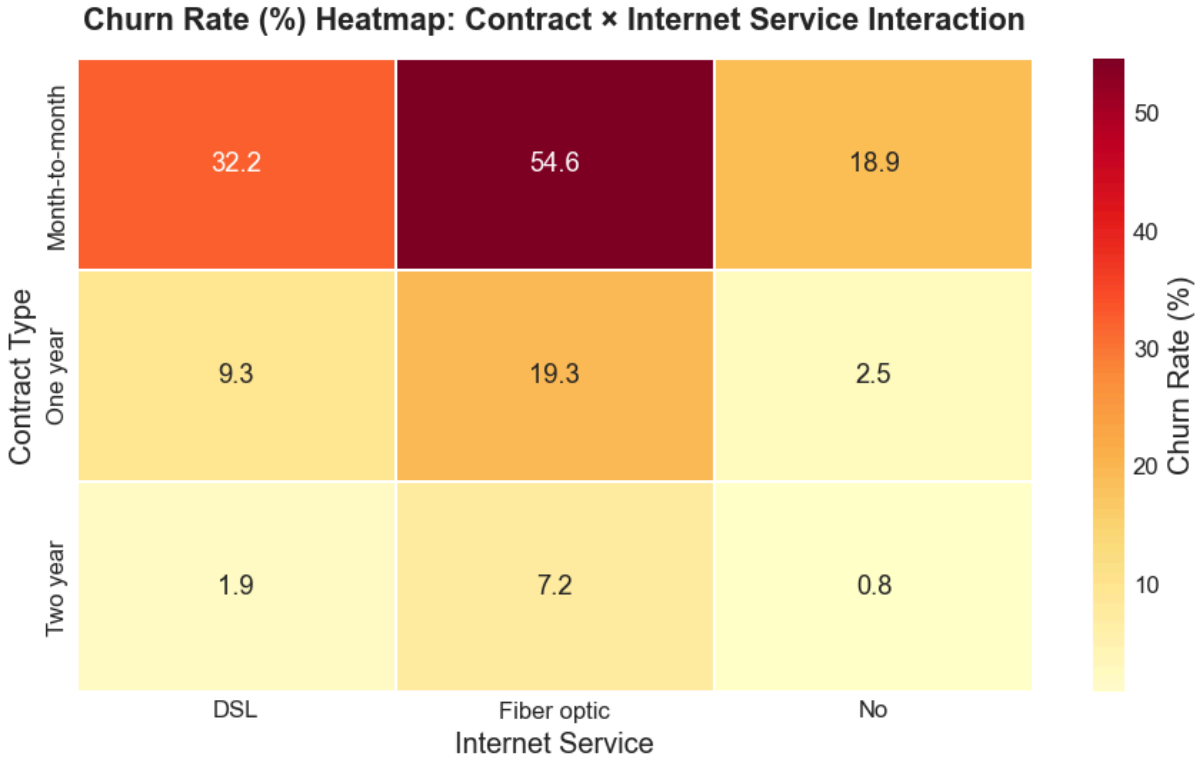
# Multi-variable interaction: Contract x Internet Service Churn Heatmap
plt.figure(figsize=(9, 4.8))
interaction = df.pivot_table(index='Contract', columns='InternetService', values='C
sns.heatmap(interaction[['DSL', 'Fiber optic', 'No']].loc[['Month-to-month', 'One y
            annot=True, fmt=".1f", cmap="YlOrRd", cbar_kws={'label': 'Churn Rate (%
plt.title('Churn Rate (%) Heatmap: Contract × Internet Service Interaction', fontwe
plt.xlabel('Internet Service')
plt.ylabel('Contract Type')
plt.show()
```

```
Internet Service Type vs Churn Contingency Table:
```

| Churn | No | Yes | Total | Churn Rate (%) |
|---|---|---|---|---|
| **InternetService** | | | | |
| **DSL** | 1,957 | 459 | 2,416 | 19.00% |
| **Fiber optic** | 1,799 | 1,297 | 3,096 | 41.89% |
| **No** | 1,407 | 113 | 1,520 | 7.43% |
| **Total** | 5,163 | 1,869 | 7,032 | nan% |

```
Chi-Square Test (Internet Service vs Churn): Chi2 = 728.70 | p-value = 5.8312e-159 | Cramér's V = 0.3219
```




In [ ]:
```
# ============================================================
# 3.3 TENURE VS MONTHLY CHARGES DISTRIBUTIONS & CORRELATIONS
# ============================================================
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```python
# Bivariate Scatter with Churn Hue
sns.scatterplot(
    data=df,
    x='tenure',
    y='MonthlyCharges',
    hue='Churn',
    palette=CHURN_PALETTE,
    alpha=0.6,
    s=35,
    ax=axes[0]
)
axes[0].set_title('Tenure vs. Monthly Charges (Stratified by Churn)', fontweight='b
axes[0].set_xlabel('Tenure (Months)')
axes[0].set_ylabel('Monthly Charges ($)')
axes[0].axvline(x=12, color='gray', linestyle='--', alpha=0.7, label='1-Year Thresh
axes[0].axhline(y=70, color='gray', linestyle=':', alpha=0.7, label='High-Charge Th
axes[0].legend(title='Churn', frameon=True)

# Hexbin Density Plot
hb = axes[1].hexbin(df['tenure'], df['MonthlyCharges'], gridsize=30, cmap='Blues',
axes[1].set_title('Subscriber Density Concentration (Hexbin)', fontweight='bold')
axes[1].set_xlabel('Tenure (Months)')
axes[1].set_ylabel('Monthly Charges ($)')
fig.colorbar(hb, ax=axes[1], label='Customer Count')

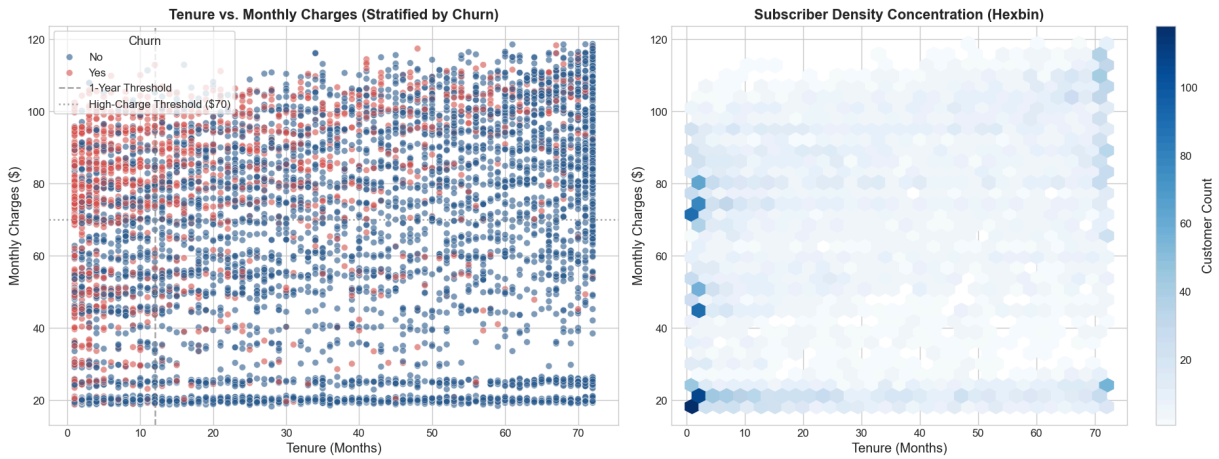
plt.tight_layout()
plt.show()

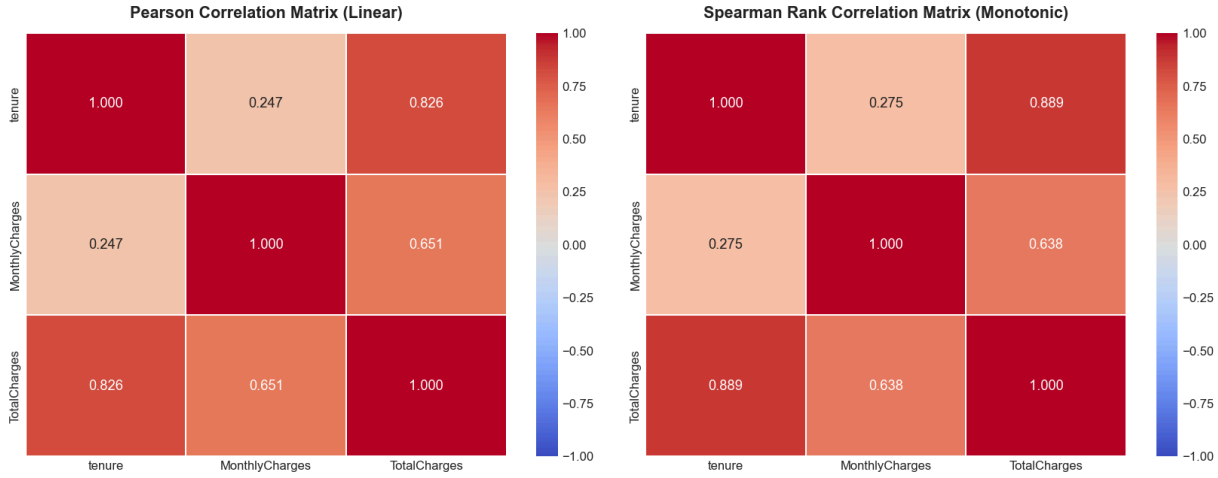
# Continuous Attribute Correlation Heatmaps (Pearson & Spearman)
cont_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
corr_pearson = df[cont_cols].corr(method='pearson')
corr_spearman = df[cont_cols].corr(method='spearman')

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
sns.heatmap(corr_pearson, annot=True, fmt=".3f", cmap='coolwarm', vmin=-1, vmax=1,
axes[0].set_title('Pearson Correlation Matrix (Linear)', fontweight='bold', pad=12)

sns.heatmap(corr_spearman, annot=True, fmt=".3f", cmap='coolwarm', vmin=-1, vmax=1,
axes[1].set_title('Spearman Rank Correlation Matrix (Monotonic)', fontweight='bold'

plt.tight_layout()
plt.show()
```






---

# 4. Advanced Feature Engineering (Merged from Preprocessing Pipeline)

## Engineered Domain Features:

1. **`TotalServices`**: Aggregates the total number of active services utilized by the customer across Phone, MultipleLines, Internet, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, and StreamingMovies.
2. **`LifetimeValue`**: Economic proxy capturing cumulative revenue potential:

$$\text{LifetimeValue} = \text{MonthlyCharges} \times \text{tenure}$$

3. **`AvgMonthlySpend`**: Effective monthly historical spend run-rate:

$$\text{AvgMonthlySpend} = \frac{\text{TotalCharges}}{\text{tenure}}$$

4. **`HasInternet`**: Binary indicator designating whether the customer subscribes to an internet infrastructure ( `InternetService != "No"` ).
5. **`ProtectionScore`**: Bundled cybersecurity and technical care index:

$$\text{ProtectionScore} = \text{OnlineSecurity} + \text{DeviceProtection} + \text{TechSupport}$$

In [ ]:
```
# ============================================================
# FEATURE ENGINEERING
# ============================================================

# ------------------------------------------------------------
# 1. Total Services
# Count the number of subscribed services across all options
# ------------------------------------------------------------
service_columns = [
    "PhoneService",
    "MultipleLines",
```

```
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies"
]
df["TotalServices"] = 0
for col in service_columns:
    df["TotalServices"] += df[col].isin(["Yes"]).astype(int)
print("✓ TotalServices created")

# Base services alternative validation (includes base internet connection)
df["TotalServices"] = (
    (df["PhoneService"] == "Yes").astype(int)
    + (df["MultipleLines"] == "Yes").astype(int)
    + (df["InternetService"] != "No").astype(int)
    + (df["OnlineSecurity"] == "Yes").astype(int)
    + (df["OnlineBackup"] == "Yes").astype(int)
    + (df["DeviceProtection"] == "Yes").astype(int)
    + (df["TechSupport"] == "Yes").astype(int)
    + (df["StreamingTV"] == "Yes").astype(int)
    + (df["StreamingMovies"] == "Yes").astype(int)
)

# ------------------------------------------------------------
# 2. Customer Lifetime Value (Proxy)
# ------------------------------------------------------------
df["LifetimeValue"] = (
    df["MonthlyCharges"] *
    df["tenure"]
)
print("✓ LifetimeValue created")

# ------------------------------------------------------------
# 3. Average Monthly Spend
# ------------------------------------------------------------
df["AvgMonthlySpend"] = (
    df["TotalCharges"] /
    df["tenure"]
)
print("✓ AvgMonthlySpend created")

# ------------------------------------------------------------
# 4. Internet User
# ------------------------------------------------------------
df["HasInternet"] = (
    df["InternetService"] != "No"
).astype(int)
print("✓ HasInternet created")

# ------------------------------------------------------------
# 5. Protection Score
# ------------------------------------------------------------
protection_columns = [
    "OnlineSecurity",
```

```
    "DeviceProtection",
    "TechSupport"
]
df["ProtectionScore"] = 0
for col in protection_columns:
    df["ProtectionScore"] += (
        df[col] == "Yes"
    ).astype(int)
print("✓ ProtectionScore created")

# Display first 5 rows of engineered features
display(df[['tenure', 'MonthlyCharges', 'TotalCharges', 'TotalServices', 'LifetimeV
```

```
✓ TotalServices created
✓ LifetimeValue created
✓ AvgMonthlySpend created
✓ HasInternet created
✓ ProtectionScore created
```

| | tenure | MonthlyCharges | TotalCharges | TotalServices | LifetimeValue | AvgMonthlySpend | H |
|---|---|---|---|---|---|---|---|
| **0** | 1 | 29.85 | 29.85 | 2 | 29.85 | 29.850000 | |
| **1** | 34 | 56.95 | 1889.50 | 4 | 1936.30 | 55.573529 | |
| **2** | 2 | 53.85 | 108.15 | 4 | 107.70 | 54.075000 | |
| **3** | 45 | 42.30 | 1840.75 | 4 | 1903.50 | 40.905556 | |
| **4** | 2 | 70.70 | 151.65 | 2 | 141.40 | 75.825000 | |

---

# 5. Binary Encoding, Correlation Analysis & One-Hot Encoding

## Encoding Strategy:

1. **Binary Mapping**: Encode binary categorical attributes ( `gender` , `Partner` , `Dependents` , `PhoneService` , `PaperlessBilling` , `Churn` ) into $[0, 1]$ numerical flags.
2. **Correlation Heatmap**: Inspect linear dependencies across all numerical and binary features before full dummy expansion.
3. **One-Hot Encoding**: Apply `pd.get_dummies` with `drop_first=True, dtype=int` on multi-class categorical predictors ( `MultipleLines` , `InternetService` , `OnlineSecurity` , `OnlineBackup` , `DeviceProtection` , `TechSupport` , `StreamingTV` , `StreamingMovies` , `Contract` , `PaymentMethod` ).
4. **Dataset Export**: Save the resulting $(7032, 36)$ clean matrix to `telco_preprocessed.csv` .

In [ ]:

```
# ============================================================
# LABEL / BINARY ENCODING
# ============================================================
binary_mapping = {
    "Yes": 1,
    "No": 0,
    "Male": 1,
    "Female": 0
}

binary_columns = [
    "gender",
    "Partner",
    "Dependents",
    "PhoneService",
    "PaperlessBilling",
    "Churn"
]

for col in binary_columns:
    df[col] = df[col].map(binary_mapping)
print("✓ Binary columns encoded")

# ============================================================
# CORRELATION HEATMAP (NUMERIC & BINARY ATTRIBUTES)
# ============================================================
plt.figure(figsize=(12, 8))
corr = df.corr(numeric_only=True)
sns.heatmap(corr, cmap="coolwarm", center=0, annot=False)
plt.title("Correlation Heatmap: Numerical & Binary Engineered Features", fontweight
plt.show()

# ============================================================
# ONE-HOT ENCODING
# ============================================================
multi_columns = [
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "Contract",
    "PaymentMethod"
]

df = pd.get_dummies(
    df,
    columns=multi_columns,
    drop_first=True,
    dtype=int
)
print("✓ One-Hot Encoding completed")
```

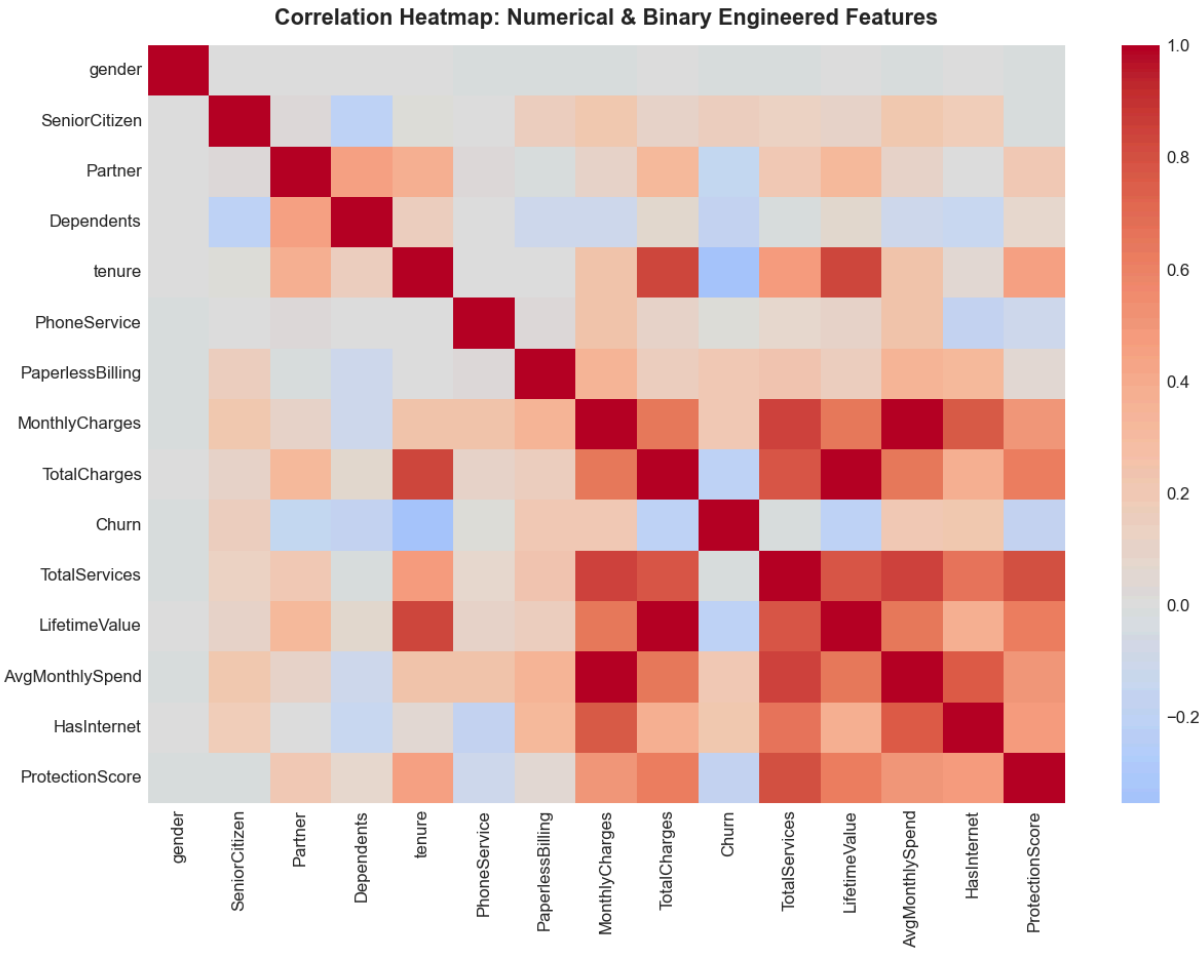
```
print("\nFinal Preprocessed Dataset Dimensions (Rows, Columns):", df.shape)
print("\nFirst 5 Records of Preprocessed Dataset:")
display(df.head())

print("\nFinal Schema Information:")
df.info()

print("\nTarget Variable ('Churn') Distribution:")
print(df["Churn"].value_counts())
print(df["Churn"].value_counts(normalize=True) * 100)

# ============================================================
# SAVE PREPROCESSED DATASET
# ============================================================
df.to_csv("telco_preprocessed.csv", index=False)
print("✓ Preprocessed dataset successfully saved to 'telco_preprocessed.csv'!")
```

✓ Binary columns encoded




✓ One-Hot Encoding completed

Final Preprocessed Dataset Dimensions (Rows, Columns): (7032, 36)

First 5 Records of Preprocessed Dataset:

| | gender | SeniorCitizen | Partner | Dependents | tenure | PhoneService | PaperlessBilling | Mont |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 1 | 0 | 1 | 0 | 1 | |
| **1** | 1 | 0 | 0 | 0 | 34 | 1 | 0 | |
| **2** | 1 | 0 | 0 | 0 | 2 | 1 | 1 | |
| **3** | 1 | 0 | 0 | 0 | 45 | 0 | 0 | |
| **4** | 0 | 0 | 0 | 0 | 2 | 1 | 1 | |

5 rows × 36 columns

```
Final Schema Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7032 entries, 0 to 7031
Data columns (total 36 columns):
 #   Column                                   Non-Null Count  Dtype
---  ------                                   --------------  -----
 0   gender                                   7032 non-null   int64
 1   SeniorCitizen                            7032 non-null   int64
 2   Partner                                  7032 non-null   int64
 3   Dependents                               7032 non-null   int64
 4   tenure                                   7032 non-null   int64
 5   PhoneService                             7032 non-null   int64
 6   PaperlessBilling                         7032 non-null   int64
 7   MonthlyCharges                           7032 non-null   float64
 8   TotalCharges                             7032 non-null   float64
 9   Churn                                    7032 non-null   int64
 10  TotalServices                            7032 non-null   int32
 11  LifetimeValue                            7032 non-null   float64
 12  AvgMonthlySpend                          7032 non-null   float64
 13  HasInternet                              7032 non-null   int32
 14  ProtectionScore                          7032 non-null   int64
 15  MultipleLines_No phone service           7032 non-null   int32
 16  MultipleLines_Yes                        7032 non-null   int32
 17  InternetService_Fiber optic              7032 non-null   int32
 18  InternetService_No                       7032 non-null   int32
 19  OnlineSecurity_No internet service       7032 non-null   int32
 20  OnlineSecurity_Yes                       7032 non-null   int32
 21  OnlineBackup_No internet service         7032 non-null   int32
 22  OnlineBackup_Yes                         7032 non-null   int32
 23  DeviceProtection_No internet service     7032 non-null   int32
 24  DeviceProtection_Yes                     7032 non-null   int32
 25  TechSupport_No internet service          7032 non-null   int32
 26  TechSupport_Yes                          7032 non-null   int32
 27  StreamingTV_No internet service          7032 non-null   int32
 28  StreamingTV_Yes                          7032 non-null   int32
 29  StreamingMovies_No internet service      7032 non-null   int32
 30  StreamingMovies_Yes                      7032 non-null   int32
 31  Contract_One year                        7032 non-null   int32
 32  Contract_Two year                        7032 non-null   int32
 33  PaymentMethod_Credit card (automatic)    7032 non-null   int32
 34  PaymentMethod_Electronic check           7032 non-null   int32
 35  PaymentMethod_Mailed check               7032 non-null   int32
dtypes: float64(4), int32(23), int64(9)
memory usage: 1.3 MB

Target Variable ('Churn') Distribution:
Churn
0    5163
1    1869
Name: count, dtype: int64
Churn
0    73.421502
1    26.578498
Name: proportion, dtype: float64
✓ Preprocessed dataset successfully saved to 'telco_preprocessed.csv'!
```

---

# 6. Interactive Interface for Data Inspection & Parameter Tuning

## Evaluation Rubric Compliance:

> ***"provide an interface to load dataset, training, parameter adjustment and for visualization."***

The widget control interface below enables interactive cohort exploration:

- **Contract Type & Internet Service Selectors**: Filter subscribers dynamically.
- **Tenure & Monthly Charges Range Sliders**: Adjust observation bounds in real time.
- **Real-Time KPI Scorecards**: Immediate calculation of filtered cohort count, churn rate, mean tenure, and mean spend.
- **Dynamic 2-Panel Plots**: Real-time rendering of bivariate scatter plots and cohort churn bars.

In [ ]:
```
# Interactive GUI Control Dashboard using ipywidgets
# Load unencoded copy for clean widget category names
df_ui = pd.read_csv(DATASET_PATH)
df_ui['TotalCharges'] = pd.to_numeric(df_ui['TotalCharges'].replace(" ", np.nan))
df_ui.dropna(subset=['TotalCharges'], inplace=True)

contract_opts = ['All'] + list(df_ui['Contract'].unique())
internet_opts = ['All'] + list(df_ui['InternetService'].unique())

w_contract = widgets.Dropdown(options=contract_opts, value='All', description='Cont
w_internet = widgets.Dropdown(options=internet_opts, value='All', description='Inte
w_tenure = widgets.IntRangeSlider(
    value=[0, int(df_ui['tenure'].max())],
    min=0, max=int(df_ui['tenure'].max()), step=1,
    description='Tenure (Mo):', style={'description_width': '90px'}, layout=widgets
)
w_charges = widgets.FloatRangeSlider(
    value=[float(df_ui['MonthlyCharges'].min()), float(df_ui['MonthlyCharges'].max(
    min=float(df_ui['MonthlyCharges'].min()), max=float(df_ui['MonthlyCharges'].max
    description='Charges ($):', style={'description_width': '90px'}, layout=widgets
)

ui_out = widgets.Output()

def update_ui(change=None):
    with ui_out:
        clear_output(wait=True)
        sub = df_ui.copy()
        if w_contract.value != 'All':
            sub = sub[sub['Contract'] == w_contract.value]
        if w_internet.value != 'All':
            sub = sub[sub['InternetService'] == w_internet.value]
```

```
        sub = sub[(sub['tenure'] >= w_tenure.value[0]) & (sub['tenure'] <= w_tenure
                  (sub['MonthlyCharges'] >= w_charges.value[0]) & (sub['MonthlyChar

        n = len(sub)
        if n == 0:
            print("No records match the active filters.")
            return

        c_count = (sub['Churn'] == 'Yes').sum()
        c_rate = (c_count / n) * 100

        banner = f"""
        <div style="display: flex; gap: 15px; margin-bottom: 12px;">
            <div style="background-color: #f0f4f8; padding: 10px 18px; border-radiu
                <span style="font-size: 11px; color: #555;">COHORT SIZE</span><br>
                <b style="font-size: 18px; color: #1a365d;">{n:,}</b>
            </div>
            <div style="background-color: #fdf0f0; padding: 10px 18px; border-radiu
                <span style="font-size: 11px; color: #555;">CHURN RATE</span><br>
                <b style="font-size: 18px; color: #c53030;">{c_rate:.2f}%</b> ({c_c
            </div>
            <div style="background-color: #f0fdf4; padding: 10px 18px; border-radiu
                <span style="font-size: 11px; color: #555;">MEAN TENURE</span><br>
                <b style="font-size: 18px; color: #22543d;">{sub['tenure'].mean():.
            </div>
            <div style="background-color: #fefcbf; padding: 10px 18px; border-radiu
                <span style="font-size: 11px; color: #555;">MEAN CHARGE</span><br>
                <b style="font-size: 18px; color: #744210;">${sub['MonthlyCharges']
            </div>
        </div>
        """
        display(HTML(banner))

        fig, axes = plt.subplots(1, 2, figsize=(15, 4.8))
        sns.scatterplot(data=sub, x='tenure', y='MonthlyCharges', hue='Churn', pale
        axes[0].set_title(f'Filtered Cohort Scatter Plot (N={n:,})', fontweight='bo
        axes[0].set_xlabel('Tenure (Months)')
        axes[0].set_ylabel('Monthly Charges ($)')

        p = sub['Churn'].value_counts(normalize=True) * 100
        bars = axes[1].bar(['No Churn', 'Churned'], [p.get('No', 0), p.get('Yes', 0
        axes[1].set_title('Cohort Churn Rate (%)', fontweight='bold')
        axes[1].set_ylabel('Percentage (%)')
        axes[1].set_ylim(0, 105)
        for b in bars:
            h = b.get_height()
            axes[1].text(b.get_x() + b.get_width()/2., h + 2, f"{h:.1f}%", ha='cent

        plt.tight_layout()
        plt.show()

w_contract.observe(update_ui, names='value')
w_internet.observe(update_ui, names='value')
w_tenure.observe(update_ui, names='value')
w_charges.observe(update_ui, names='value')
```

```
controls = widgets.VBox([
    widgets.HTML("<b>Interactive Parameter Adjustment & Exploration Interface:</b>"
    widgets.HBox([w_contract, w_internet]),
    widgets.HBox([w_tenure, w_charges])
])

display(controls)
display(ui_out)
update_ui()
```